

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO HÀNG TUẦN MÔN
THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn	: TS. Kim Ngọc Bách
Sinh viên thực hiện	: Mã sinh viên
Nguyễn Việt Quang	: B23DCCN691

Hà Nội – 2026

I. Tổng quan về Quantization và Pruning trong nén mô hình học sâu

1. Mở đầu

Trong bối cảnh các mô hình học sâu ngày càng có quy mô lớn, việc triển khai chúng trên các thiết bị có tài nguyên tính toán hạn chế hoặc phục vụ suy luận ở quy mô lớn đặt ra những thách thức đáng kể về bộ nhớ, tốc độ và chi phí năng lượng. Hai kỹ thuật nén mô hình được nghiên cứu và ứng dụng rộng rãi nhất hiện nay là Quantization (lượng tử hóa) và Pruning (cắt tỉa mô hình). Mỗi kỹ thuật tiếp cận bài toán tối ưu hóa theo hướng khác nhau nhưng đều hướng đến mục tiêu chung là giảm chi phí suy luận mà vẫn giữ được chất lượng mô hình ở mức chấp nhận được.

2. Quantization (Lượng tử hóa mô hình)

2.1. Khái niệm và nguyên lý

Quantization là kỹ thuật giảm độ chính xác số học trong biểu diễn trọng số và activation của mô hình, từ định dạng dấu phẩy động 32-bit (FP32) xuống các định dạng có độ phân giải thấp hơn như INT8, INT4 hay FP16. Về mặt kỹ thuật, quá trình này ánh xạ một tập giá trị liên tục trong miền FP32 sang một tập hữu hạn các giá trị nguyên, sử dụng hai tham số là hệ số tỉ lệ (scale factor) và điểm không (zero-point) để duy trì khả năng chuyển đổi ngược. Kết quả là mô hình có kích thước nhỏ hơn, yêu cầu băng thông bộ nhớ thấp hơn, và trên phần cứng hỗ trợ phép tính nguyên (như CPU với tập lệnh VNNI hoặc NPU), tốc độ suy luận có thể được cải thiện đáng kể.

2.2. Các phương pháp Quantization chính

Post-Training Quantization (PTQ) là nhóm phương pháp áp dụng quantization sau khi mô hình đã được huấn luyện hoàn chỉnh, không yêu cầu quá trình huấn luyện lại. PTQ chia thành hai hướng chính tùy theo thời điểm tính toán tham số lượng tử hóa. Dynamic Quantization tính toán scale factor và zero-point ngay trong thời gian suy luận dựa trên phân phối thực tế của activation đầu vào. Phương pháp này không yêu cầu tập dữ liệu hiệu chỉnh (calibration dataset), dễ triển khai và phù hợp với các mô hình xử lý chuỗi như LSTM hay Transformer, nhưng phát sinh chi phí tính toán thêm trong mỗi lần suy luận do phải xác định lại khoảng giá trị theo từng đầu vào. Static Quantization ngược lại, xác định các tham số lượng tử hóa một lần duy nhất thông qua một bước hiệu chỉnh sử dụng tập dữ liệu đại diện, sau đó cố định các tham số này cho mọi lần suy luận. Cách tiếp cận này cho hiệu năng cao hơn nhưng đòi hỏi tập dữ liệu hiệu chỉnh phải phản ánh được phân phối đầu vào thực tế.

Quantization-Aware Training (QAT) là phương pháp tích hợp quá trình mô phỏng quantization trực tiếp vào vòng lặp huấn luyện. Cụ thể, trong mỗi lượt forward pass, mô hình giả lập phép tính độ chính xác thấp bằng cách chèn các nút "fake quantization" vào đồ thị tính toán. Mặc dù trọng số vẫn được lưu và cập nhật ở FP32, mô hình được phơi bày với sai số lượng tử hóa trong quá trình tối ưu, từ đó học cách tìm các trọng số có khả năng chịu đựng sai số này tốt hơn. QAT thường cho độ chính xác cao hơn PTQ nhưng đi kèm với chi phí tính toán lớn hơn do phải huấn luyện lại mô hình.

2.3. Đánh đổi và lưu ý khi áp dụng

Một điểm quan trọng cần nhấn mạnh là tốc độ suy luận của mô hình INT8 không tự động cao hơn FP32 trong mọi trường hợp. Lợi ích thực sự của quantization phụ thuộc mạnh vào kiến trúc phần cứng: INT8 chỉ cho thấy speedup đáng kể khi backend hỗ trợ phép tính nguyên hiệu quả. Trên CPU không có tập lệnh tối ưu cho INT8, hoặc với các mô hình sinh chuỗi tự hồi quy (autoregressive), chi phí chuyển đổi giữa hai miền số học (quantize/dequantize) và các overhead khác có thể lớn hơn lợi ích thu được, dẫn đến kết quả ngược chiều so với kỳ vọng. Ngoài ra, chất lượng mô hình sau quantization phụ thuộc vào độ nhạy cảm của từng kiến trúc với sai số lượng tử hóa, đòi hỏi đánh giá thực nghiệm cẩn thận thay vì giả định rằng INT8 luôn nhanh hơn và tốt hơn.

3. Pruning (Cắt tỉa mô hình)

3.1. Khái niệm và nguyên lý

Pruning là kỹ thuật nén mô hình dựa trên quan sát rằng các mạng nơ-ron thường có một lượng lớn tham số dư thừa (redundant parameters) không đóng góp đáng kể vào hiệu năng cuối cùng. Ý tưởng cốt lõi là xác định và loại bỏ những tham số này, từ đó giảm số lượng phép tính và dung lượng lưu trữ cần thiết. Pruning thường được thực hiện theo chu trình: huấn luyện mô hình ban đầu, xác định và loại bỏ các tham số kém quan trọng nhất, sau đó tinh chỉnh lại mô hình đã cắt tỉa để phục hồi độ chính xác. Quá trình này có thể lặp lại nhiều vòng (iterative pruning).

3.2. Unstructured Pruning và Structured Pruning

Unstructured pruning (cắt tỉa phi cấu trúc) loại bỏ các trọng số riêng lẻ có giá trị tuyệt đối nhỏ, không quan tâm đến vị trí của chúng trong ma trận trọng số. Kết quả là mô hình có dạng ma trận thưa (sparse), về mặt lý thuyết có thể đạt tỉ lệ nén cao nhưng trên thực tế đòi hỏi hỗ trợ phần cứng và thư viện tính toán thưa đặc biệt để chuyển hóa thành speedup thực sự. Trên phần

cứng thông thường, matrix thưa từ unstructured pruning thường không cho tốc độ suy luận nhanh hơn đáng kể so với mô hình dày đặc ban đầu.

Structured pruning (cắt tỉa có cấu trúc) loại bỏ toàn bộ các đơn vị có cấu trúc như filter trong lớp tích chập, attention head trong Transformer, hay toàn bộ một lớp mạng. Cách tiếp cận này tạo ra mô hình vẫn có cấu trúc dày đặc nhưng nhỏ hơn, thân thiện với phần cứng thông thường và có thể được tăng tốc trực tiếp mà không cần hỗ trợ đặc biệt. Đây là lý do structured pruning được ưu tiên trong nhiều ứng dụng triển khai thực tế, đặc biệt trong lĩnh vực nén Large Language Models và Vision Transformers.

3.3. Khi nào nên cắt tỉa và tiêu chí xác định tầm quan trọng

Việc xác định tham số nào cần loại bỏ là bài toán trung tâm của pruning. Tiêu chí phổ biến nhất là magnitude-based pruning, trong đó các trọng số có giá trị tuyệt đối nhỏ hơn một ngưỡng nhất định được coi là không quan trọng và bị loại bỏ. Các tiêu chí phức tạp hơn có thể dựa trên gradient, thông tin Fisher, hay hàm loss. Về thời điểm thực hiện, pruning có thể được áp dụng trên mô hình đã huấn luyện hoàn chỉnh (post-training pruning) để đơn giản hóa triển khai, hoặc tích hợp trong quá trình huấn luyện để mô hình học cách thích nghi với việc bị cắt tỉa. Pruning cũng có thể kết hợp với quantization để đạt mức nén tối đa, tận dụng lợi thế của cả hai kỹ thuật.

4. So sánh tổng quan Quantization và Pruning

Hai kỹ thuật này bổ sung cho nhau hơn là cạnh tranh. Quantization thay đổi độ chính xác biểu diễn số học của toàn bộ mô hình, phù hợp khi mục tiêu chính là giảm băng thông bộ nhớ và tận dụng phần cứng tối ưu hóa cho số học nguyên. Pruning thay đổi cấu trúc và mật độ của mô hình bằng cách loại bỏ tham số dư thừa, phù hợp khi mục tiêu là giảm số lượng phép tính (FLOPs) và kích thước mô hình. Trong thực tiễn, cả hai kỹ thuật đều đòi hỏi đánh giá thực nghiệm kỹ lưỡng vì hiệu quả phụ thuộc mạnh vào kiến trúc mô hình cụ thể, bài toán đang giải quyết, và đặc biệt là phần cứng triển khai mục tiêu.

Tiêu chí	Quantization	Pruning
Nguyên lý	Giảm độ chính xác số học (FP32 → INT8)	Loại bỏ tham số dư thừa
Yêu cầu huấn luyện lại	Không (PTQ) hoặc có (QAT)	Thường cần fine-tune sau cắt tỉa
Thay đổi kiến trúc	Không	Có (structured) hoặc không (unstructured)

Tiêu chí	Quantization	Pruning
Hỗ trợ phần cứng	Cần backend INT8 phù hợp	Structured không cần hỗ trợ đặc biệt
Mức nén điển hình	4x (FP32 $\rightarrow$ INT8)	Tùy tỉ lệ cắt tỉa, thường 30–80%

Bảng 1. So sánh tổng quan giữa Quantization và Pruning

II. Quantization mô hình BLIP – Thực nghiệm và Đánh giá kết quả

1. Mở đầu

Tiếp nối kết quả của tuần trước với mô hình v3 đạt BLEU-4 là 33.61, ROUGE-L là 53.33 và METEOR là 40.11 trên tập test 2.001 ảnh, tuần này tập trung vào việc tối ưu hóa chi phí suy luận của mô hình thông qua kỹ thuật quantization. Khi mô hình BLIP đã đạt chất lượng caption đủ tốt, câu hỏi tiếp theo đặt ra là liệu có thể giảm tài nguyên tính toán cần thiết mà không hy sinh đáng kể chất lượng đầu ra hay không. Đây là bước quan trọng hướng tới mục tiêu triển khai hệ thống trên thiết bị phổ thông phục vụ người khiếm thị.

Tuy nhiên, kết quả thực nghiệm trong tuần này cho thấy quantization là một bài toán phức tạp hơn so với kỳ vọng ban đầu, đặt ra nhiều câu hỏi quan trọng về điều kiện áp dụng và lựa chọn backend phù hợp.

2. Mục tiêu thực hiện

Mục tiêu của phần thực nghiệm này là kiểm tra khả năng lượng tử hóa mô hình sinh mô tả ảnh BLIP nhằm giảm chi phí suy luận, đồng thời đánh giá mức độ ảnh hưởng của quantization lên tốc độ suy luận, throughput và chất lượng caption sinh ra.

Phạm vi công việc đã thực hiện bao gồm bốn hạng mục chính: xây dựng baseline FP32 để làm mốc so sánh; thực hiện PyTorch dynamic quantization INT8; chuẩn bị pipeline ONNX FP32 và ONNX INT8; và xây dựng notebook so sánh kết quả giữa các phiên bản.

3. Môi trường và thiết lập thực hiện

Toàn bộ thực nghiệm được thực hiện trên nền tảng Kaggle Notebook với cùng cấu hình môi trường đã sử dụng ở các tuần trước: Python 3.12, PyTorch, Transformers của Hugging Face và thư viện onnxruntime để hỗ trợ pipeline ONNX. Mô hình đầu vào là checkpoint tốt nhất của v3 đã lưu tại thư mục saved_models_v3. Tập test sử dụng 200 ảnh được lấy mẫu từ cleaned_test.csv với nhiều caption tham chiếu được gom theo từng ảnh.

4. Nội dung thực hiện chi tiết

4.1. Xây dựng baseline FP32

Bước đầu tiên là xây dựng một notebook benchmark đầy đủ cho mô hình gốc ở định dạng FP32. Pipeline này nạp mô hình BLIP từ `saved_models_v3`, chuẩn bị tập test từ `cleaned_test.csv` với cơ chế gom nhiều caption tham chiếu theo từng ảnh, sau đó sinh caption bằng hàm `model.generate()` với các tham số beam search đã được chuẩn hóa từ tuần trước. Quá trình benchmark đo đồng thời ba nhóm chỉ số: hiệu năng suy luận gồm average latency per batch và P95 latency; throughput đo bằng số ảnh xử lý được mỗi giây; và chất lượng caption thông qua BLEU-4, ROUGE-L và METEOR. Kết quả được lưu vào `results/baseline_fp32.json` để phục vụ so sánh.

4.2. Thực hiện PyTorch Dynamic Quantization INT8

Phiên bản INT8 được tạo ra bằng cách áp dụng `torch.quantization.quantize_dynamic()` lên mô hình FP32, lượng tử hóa toàn bộ các lớp Linear sang kiểu dữ liệu qint8. Đây là lựa chọn dynamic quantization vì phương pháp này không yêu cầu tập hiệu chỉnh và phù hợp với bài toán sinh chuỗi có độ dài biến đổi. Artifact mô hình được lưu vào `model_quantized/pytorch_int8/` để đảm bảo tái sử dụng được. Sau đó, toàn bộ pipeline benchmark được chạy lại trên mô hình INT8 với cùng tập test và cùng cấu hình sinh caption, đảm bảo tính so sánh được giữa hai phiên bản. Kết quả được lưu vào `results/pytorch_int8.json`.

4.3. Chuẩn bị pipeline ONNX

Song song với PyTorch INT8, mô hình cũng được export sang định dạng ONNX theo hai bước: trước tiên export sang ONNX FP32, sau đó áp dụng quantization để có ONNX INT8. Hai file artifact tương ứng là `model_quantized/onnx_fp32/blip_captioning_fp32.onnx` và `model_quantized/onnx_int8/blip_captioning_int8.onnx` đã được tạo thành công. Hai notebook `onnx_export_fp32.ipynb` và `onnx_int8_quantization.ipynb` cũng được hoàn thiện để ghi lại quy trình. Tuy nhiên, theo phạm vi công việc tuần này, phần benchmark cho ONNX chưa được thực thi, nên kết quả trong báo cáo này chỉ tập trung vào baseline FP32 và PyTorch INT8.

4.4. Chuẩn hóa notebook compare_results

Notebook `compare_results.ipynb` được xây dựng và điều chỉnh để tổng hợp kết quả từ nhiều phiên bản mô hình, đọc đúng các file JSON kết quả, hiển thị metadata benchmark và phân biệt rõ backend, precision và thiết bị tính toán. Đây là công cụ quan trọng để duy trì tính nhất quán trong việc so sánh và sẽ được mở rộng khi bổ sung kết quả ONNX ở tuần tiếp theo.

5. Kết quả thực nghiệm

5.1. Kết quả baseline FP32

Pipeline benchmark được chạy trên 200 ảnh test, thu được bộ số liệu cơ sở như sau:

Chỉ số	Giá trị
Average latency per batch	10.03 giây
P95 latency per batch	14.01 giây
Throughput	1.596 ảnh/giây
BLEU-4	24.50
ROUGE-L	50.55
METEOR	35.88

Bảng 2. Kết quả benchmark baseline FP32 trên 200 ảnh test

5.2. Kết quả PyTorch INT8

Sau khi áp dụng dynamic quantization, mô hình INT8 được benchmark trên cùng tập test:

Chỉ số	FP32 (Baseline)	PyTorch INT8	Thay đổi
Average latency/batch	10.03 giây	16.39 giây	+63.4%
P95 latency/batch	14.01 giây	20.05 giây	+43.1%
Throughput	1.596 ảnh/giây	0.488 ảnh/giây	-69.4%
BLEU-4	24.50	20.29	-4.21
ROUGE-L	50.55	47.66	-2.89
METEOR	35.88	34.02	-1.86

Bảng 3. Kết quả benchmark PyTorch INT8 trên 200 ảnh test

6. Nhận xét và đánh giá

6.1. Về chất lượng caption

Sau lượng tử hóa INT8, chất lượng caption giảm nhẹ trên cả ba metric: BLEU-4 giảm 4.21 điểm (từ 24.50 xuống 20.29), ROUGE-L giảm 2.89 điểm và METEOR giảm 1.86 điểm. Mức

giảm này cho thấy dynamic quantization có ảnh hưởng nhất định đến khả năng sinh caption, nhưng chưa đến mức làm mất hoàn toàn khả năng mô tả hình ảnh. Điều này nhất quán với các nghiên cứu trong lĩnh vực, theo đó quantization thường gây ra sự suy giảm nhỏ về chất lượng, đặc biệt khi áp dụng cho các mô hình ngôn ngữ có phân phối trọng số phức tạp.

6.2. Về hiệu năng suy luận

Kết quả thực nghiệm cho thấy phiên bản PyTorch INT8 chậm hơn đáng kể so với baseline FP32: latency tăng hơn 63%, throughput giảm gần 70%. Đây là kết quả trái với kỳ vọng ban đầu và cần được phân tích kỹ. Có một số nguyên nhân kỹ thuật giải thích hiện tượng này. Thứ nhất, dynamic quantization của PyTorch tính toán scale factor và zero-point trong thời gian suy luận, phát sinh overhead trong mỗi bước sinh token của quá trình tự hồi quy. Thứ hai, phép tính nguyên INT8 chỉ thực sự nhanh hơn FP32 khi phần cứng có tập lệnh tối ưu hóa như VNNI trên Intel Cascade Lake; trên phần cứng không hỗ trợ hoặc ở batch size nhỏ, chi phí quantize/dequantize có thể lấn át lợi ích tiết kiệm bộ nhớ. Thứ ba, mô hình sinh chuỗi là bài toán đặc biệt khó tận dụng lợi ích của INT8 vì mỗi bước sinh chỉ xử lý một token, dẫn đến kích thước ma trận quá nhỏ để khai thác hiệu quả phép nhân nguyên song song.

Đây là bài học quan trọng: quantization không phải giải pháp toàn năng tự động tăng tốc. Hiệu quả của nó gắn chặt với backend, kiến trúc mô hình và đặc thù bài toán. Cần đánh giá thực nghiệm thay vì giả định lý thuyết.

7. Kết luận và định hướng tuần tiếp theo

Trong tuần này, em đã hoàn thành xây dựng pipeline benchmark đầy đủ cho quá trình quantization mô hình BLIP, bao gồm baseline FP32, PyTorch INT8 và chuẩn bị thêm pipeline ONNX. Kết quả định lượng cho thấy phiên bản PyTorch INT8 chưa đạt được mục tiêu tối ưu hóa mong muốn: chất lượng caption giảm nhẹ trong khi tốc độ suy luận lại thấp hơn đáng kể so với baseline. Tuy nhiên, về mặt nghiên cứu và thực nghiệm, phần việc này đã đạt được điều quan trọng nhất là xây dựng được một pipeline đánh giá quantization có thể tái sử dụng và rút ra kết luận có cơ sở từ dữ liệu thực tế.

Kết quả này đặt ra hai hướng cải thiện rõ ràng cho tuần tiếp theo. Hướng thứ nhất là hoàn thiện và chạy benchmark pipeline ONNX INT8, vì ONNX Runtime có khả năng tối ưu hóa đồ thị tính toán tốt hơn và có thể cho kết quả hiệu năng khả quan hơn so với PyTorch dynamic quantization. Hướng thứ hai là khảo sát Static Quantization hoặc Quantization-Aware Training như các phương án thay thế, vì những phương pháp này có khả năng bảo toàn chất lượng mô hình tốt hơn và phù hợp hơn với đặc thù bài toán sinh chuỗi.